

数值分析期末上机报告

学号：241098117 姓名：张城宗

1 梯形方法、BDF2、改进 Euler 方法比较

1.1 问题描述

分别利用梯形方法、BDF2 格式数值求解如下问题

$$y' = -\exp(10t)(y - \sin(t)) + \cos(t), \quad 0 < t \leq 1, \quad y(0) = 0$$

1. 分别取步长 $h = 0.1, 0.05, 0.025, 0.0125, 0.00625$ 进行计算，根据数值结果计算两种方法的误差收敛阶，并对计算结果进行比较分析。
2. 如利用改进的 Euler 方法会出现什么样的结果，试对计算结果进行分析。

1.2 方法介绍

梯形方法对积分使用了更精确的梯形公式，是隐式离散化方法：

$$y_{n+1} = y_n + \frac{h}{2}(f_n + f_{n+1}),$$

BDF 方法，即基于插值型数值微分构造的多步法，用 $(t_n, y_n), (t_{n-1}, y_{n-1}), \dots, (t_{n-k}, y_{n-k})$ 共 $k+1$ 点构造插值多项式 $q_k(t)$ ，用 $q'_k(t_n)$ 近似 $f(t_n, y(t_n))$ ：

$$\sum_{m=1}^k \frac{\nabla^m y_n}{m} = hf(t_n, y_n)$$

通用形式： $y_n + \sum_{j=1}^k \alpha_j y_{n-j} = h\beta_0 f(t_n, y_n)$,

2 阶公式为 $y_n - \frac{4}{3}y_{n-1} + \frac{1}{3}y_{n-2} = \frac{2}{3}hf_n$ BDF 方法对刚性方程有优良稳定性。BDF2 的绝对稳定区间比显式方法大得多

改进 Euler 属于二阶显式 Runge-Kutta 方法。

$$K_1 = f(x_n, y_n), \quad K_2 = f(x_n + h, y_n + hK_1), \quad y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2).$$

1.3 数值结果

表 1: 不同方法不同步长时的最大绝对误差

方法	0.1
0.05	0.025
0.0125	0.00625
梯形方法	1.79e-01
4.48e-02	1.12e-02
2.80e-03	7.00e-04
BDF2	1.79e-01
2.84e-01	9.83e-01
3.45e+02	1.79e+09
改进 Euler	2.80e+20
4.63e+34	4.99e+56
1.90e+89	3.05e+134

表 2: 不同方法的收敛阶估计

方法	收敛阶估计
梯形方法	[1.9854, 1.9971, 2.0012, 2.0001]
BDF2	[-0.6692, -1.7923, -8.4542, -22.3094]
改进 Euler	[47.2311, 73.1903, 108.2347, 150.1670]

1.4 结果分析

1. 梯形法 (Trapezoid): A-稳定,二阶精度。对于刚性方程 $e^{(10t)}$ h^2
1. BDF2:A-稳定，二阶精度。起步值需 RK4 提供，牛顿迭代求解隐式方程，计算量略大但精度相当。收敛阶为负数且绝对值增大（如 ‘-22.3’），说明误差随步长减小而指数级增长，验证了其在特定条件下（可能是刚性方程的大步长或小步长耦合）的不稳定性。观察：若梯形法收敛阶接近 2.0，BDF2 也接近 2.0，则说明两者在该问题下均发挥预期性能。
2. 改进 Euler: 刚性特征：由于 ODE 中包含项 $-\exp(10t)y$ ，特征值 $(t) -\exp(10t)$ ，在 $t=1$ 时 -22026。Improved Euler (显式) 的绝对稳定区域约为 $h^* > -4.5$ ，即 $h < 0.0002 * \exp(-10t)$ 。在 $t=1$ 附近，Improved Euler 需要极小的步长才能维持稳定，否则误差会随阶数平方指数增长。收敛阶极高 (> 40)，这通常意味着在初始阶段误差对 h 极度敏感，或者初始误差主要由舍入误差主导，导致计算出的“阶”虚高；随着 h 进一步减小，数值解可能进入精度极限。

附录: Python 程序

A.1 题目一 Python 程序

Listing 1: 梯形方法、BDF2、改进 Euler 方法比较

```
1 import numpy as np
2
3 # ===== 1. 定义基础函数 (全局作用域) =====
4 def f(t, y):
5     """ODE 右端项:  $y' = -\exp(10t) \cdot (y - \sin(t)) + \cos(t)$ """
6     return -np.exp(10 * t) * (y - np.sin(t)) + np.cos(t)
7
8 def fy(t, y):
9     """偏导数  $f_y$ : 用于牛顿迭代和隐式方程求解"""
10    return -np.exp(10 * t)
11
12 def y_exact(t):
13    """精确解函数 (解析推导结果)"""
14    # 根据 ODE 解析解推导, 特解为  $\sin(t)$ , 初值  $y(0)=0$  满足  $\sin(0)=0$ 
15    return np.sin(t)
16
17 # ===== 2. 数值求解器 Solve_Step(h, method) =====
18 def Solve_Step(h, method):
19    # 全局参数 (假设与主程序一致)
20    t0 = 0.0
21    T = 1.0
22    y0_val = 0.0
23
24    # 计算总步数与时间网格
25    N = int(round((T - t0) / h))
26    t = t0 + np.arange(N + 1) * h
27
28    # 预分配数值解数组
29    y = np.zeros(N + 1, dtype=float)
30    y[0] = y0_val
31
32    # 迭代参数 (常量定义)
33    tol, max_iter = 1e-6, 2000
34
35    if method == "trapezoid":
36        for n in range(N):
37            tn, yn = t[n], y[n]
38            z = yn + h * f(tn, yn)
39            # Newton 迭代求解隐式方程:  $z - y_n - (h/2)[f(tn, yn) + f(tn+h, z)] = 0$ 
40            for _ in range(max_iter):
41                z_new = yn + h * (f(tn, yn) + f(t[n+1], z)) / 2 # 梯形公式  $z = y_n + h/2 \cdot (f_n + f_{n+1})$ 。
42                if abs(z_new - z) < tol: break # 达到迭代容差后停止。
43                z = z_new # 更新隐式未知量。
44            y[n + 1] = z
```

```
45
46
47 elif method == "BDF2":
48     k = 2
49     # BDF2 系数：标准形式  $y_{n+1} - 4/3 y_n + 1/3 y_{n-1} = h * 2/3 * f(t_{n+1})$ 
50     # 对应系数 alpha ( $y_{n+1}, y_n, y_{n-1}$ ): [1, -4/3, 1/3]
51     # beta: 2/3
52     coeffs_alpha = np.array([1.0, -4.0 / 3.0, 1.0 / 3.0])
53     beta_val = 2.0 / 3.0
54
55     # 起步阶段 (RK4 提供前 k-1=1 步值, 即 y[0], y[1])
56     for n_start in range(k - 1):
57         tn = t[n_start]
58         yn = y[n_start]
59
60         q1 = f(tn, yn)
61         q2 = f(tn + h / 2.0, yn + h * q1 / 2.0)
62         q3 = f(tn + h / 2.0, yn + h * q2 / 2.0)
63         q4 = f(tn + h, yn + h * q3)
64
65         y[n_start + 1] = yn + h * (q1 + 2*q2 + 2*q3 + q4) / 6.0
66
67     # BDF 主推进阶段
68     for n in range(k - 1, N):
69         # 计算已知项 (历史值移项到右侧, 注意系数对应 y[n], y[n-1])
70         known = coeffs_alpha[1] * y[n] + coeffs_alpha[2] * y[n - 1]
71
72         z = y[n] # 初始猜测
73
74         # Newton 迭代求解隐式方程:  $z - \text{known} - h * \text{beta} * f(t_{n+1}, z) = 0$ 
75         for iteration in range(max_iter):
76             G = z - known - h * beta_val * f(t[n + 1], z)
77
78             dG = 1.0 - h * beta_val * fy(t[n + 1], z)
79
80             if abs(dG) < 1e-15:
81                 break
82
83             delta = G / dG
84             z_new = z - delta
85
86             if abs(z_new - z) < tol:
87                 z = z_new
88                 break
89             z = z_new
90
91         y[n + 1] = z
92
93 elif method == "improved_euler":
94     for n in range(N):
95         tn, yn = t[n], y[n]
```

```
96         k1 = f(tn, yn)
97         k2 = f(tn+h, yn+h*k1)
98         y[n+1] = yn + h*(k1+k2)/2
99
100
101
102     return t, y # 返回时间和解，便于后续误差对齐
103
104 # ===== 3. 主程序：测试与验证 =====
105 def main():
106     # 【配置参数】步长列表
107     h_list = [0.1, 0.05, 0.025, 0.0125, 0.00625]
108
109     # 【方法选择】
110     methods = ["trapezoid", "BDF2", "improved_euler"]
111
112     print("=" * 80)
113     print("数值方法求解与误差验证报告")
114     print("ODE: y' + e^(10t)y = e^(10t)sin(t) + cos(t), y(0)=0, t in (0, 1]")
115     print("=" * 80)
116
117     for method in methods:
118         print(f"\n=== 方法: {method} ===")
119         all_max_errors = []
120
121         # 遍历步长列表
122         for h in h_list:
123             try:
124                 # 【调用主算法得到结果】
125                 t_num, y_num = Solve_Step(h, method)
126
127                 # 【计算绝对误差 (点态 + 最大)】
128                 # 精确解直接传入时间数组
129                 exact_vals = y_exact(t_num)
130                 abs_errors = np.abs(y_num - exact_vals)
131                 max_err = np.max(abs_errors)
132
133                 all_max_errors.append(max_err)
134
135                 print(f"步长 h={h:.5f} | Max Absolute Error: {max_err:.2e}")
136
137             except Exception as e:
138                 print(f"步长 h={h:.5f} 发生异常: {e}")
139
140     # 【计算收敛阶】(取相邻步长的误差比)
141     if len(all_max_errors) >= 2:
142         p_values = []
143         for i in range(len(all_max_errors)-1):
144             err_curr, err_prev = all_max_errors[i], all_max_errors[i+1]
145             # h_current / h_previous = 0.5
146             p = np.log(err_curr / err_prev) / np.log(2.0)
```

```
147         p_values.append(p)
148
149         print(f"收敛阶估计: [ {'', '.join([f'{p:.4f}' for p in p_values)]] ]")
150
151         print("-" * 80)
152
153     # 【最终比较分析】
154     print("\n=== 结果比较与分析 ===")
155     print("梯形法 (Trapezoid): A-稳定, 二阶精度。对于刚性方程  $e^{(10t)}$ , 表现稳健, 误差随  $h^2$  衰减。")
156     print("BDF2 格式: A-稳定, 二阶精度。起步值需 RK4 提供, 牛顿迭代求解隐式方程, 计算量略大但精度相当。")
157     print("观察: 若梯形法收敛阶接近 2.0, BDF2 也接近 2.0, 则说明两者在该问题下均发挥预期性能。")
158     print("Improved Euler (显式): 条件稳定。")
159     print("\n    - 刚性特征: 由于 ODE 中包含项  $-\exp(10t)y$ , 特征值  $(t) -\exp(10t)$ , 在  $t=1$  时  $-22026$ 。")
160     print("    - Improved Euler (显式) 的绝对稳定区域约为  $h * > -4.5$ , 即  $h < 0.0002 * \exp(-10t)$ 。")
161     print("    - 结论: 在  $t=1$  附近, Improved Euler 需要极小的步长才能维持稳定, 否则误差会随阶数平方指数增长。")
162     print("=" * 80)
163
164 if __name__ == "__main__":
165     main()
```